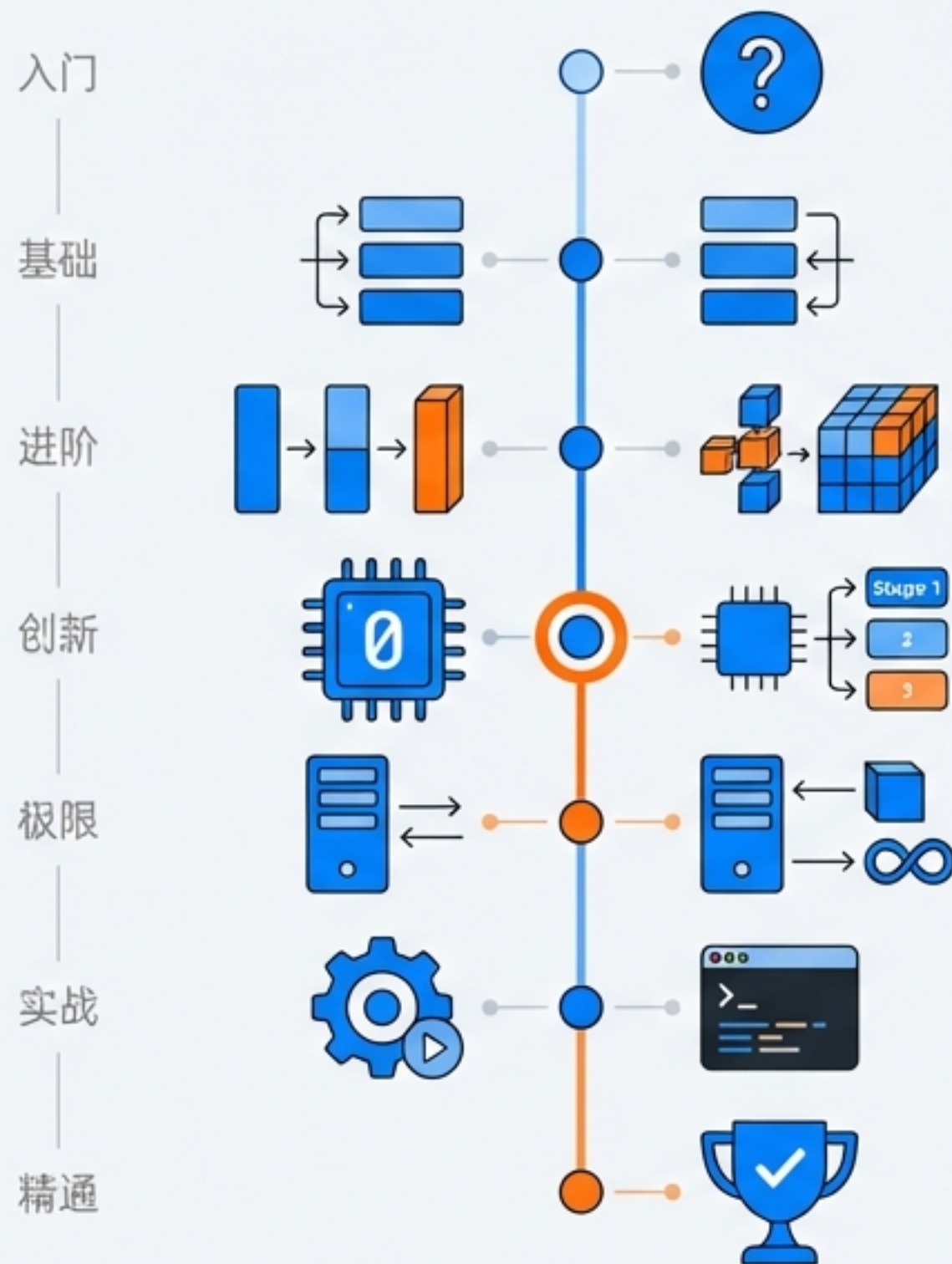




DeepSpeed大模型分布式框架

挑战每天讲透一个AI大模型知识点

本次分享大纲



1. **核心挑战：**为何需要分布式训练？
2. **基础并行策略：**数据并行（Data Parallelism）的原理与局限
3. **进阶并行策略：**模型并行（Model Parallelism）- 流水线与张量
4. **DeepSpeed核心创新：**ZeRO - 零冗余优化器详解（Stage 1-3）
5. **突破硬件极限：**ZeRO-Offload 与 ZeRO-Infinity
6. **实战演练：**DeepSpeed配置与启动
7. **总结：**如何选择最适合你的并行策略

核心挑战：AI模型的“内存墙”

如何训练一个参数量远超单张GPU显存的大模型？



大模型训练需要巨大的显存来存储模型参数、梯度、优化器状态和激活值。

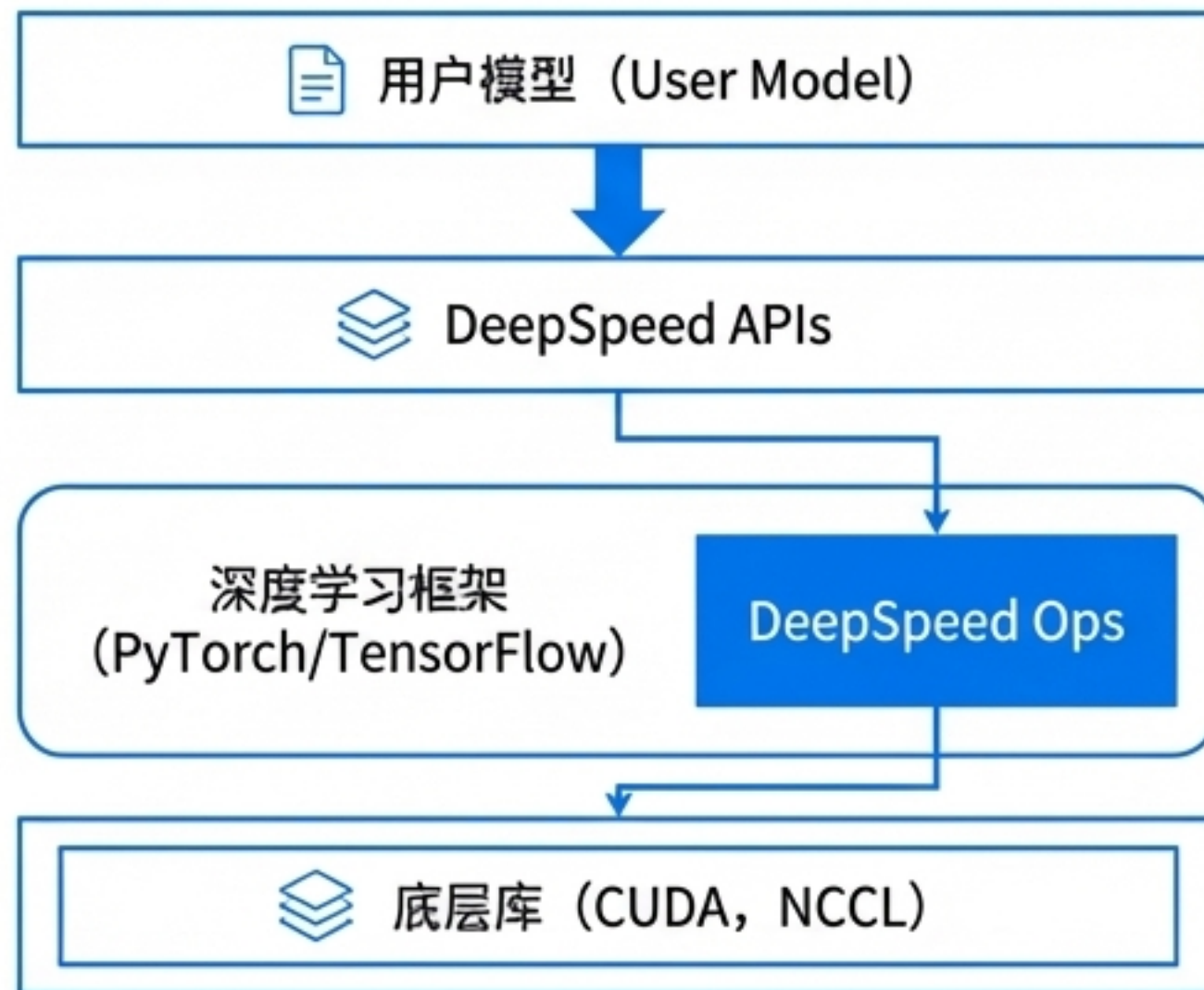
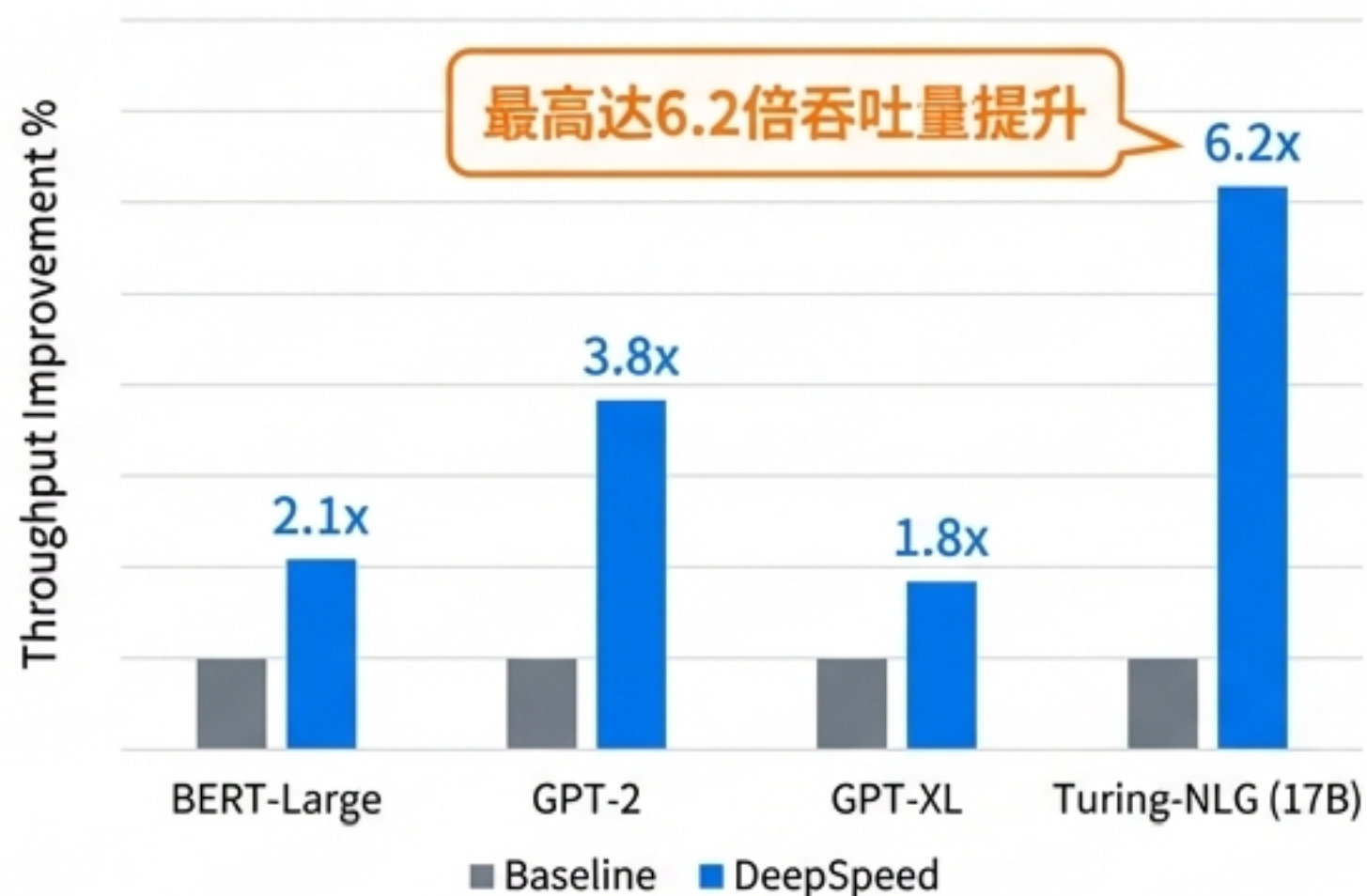
以一个1750亿参数的GPT-3模型为例，仅模型参数和优化器状态就需要数TB的内存，远超任何单张GPU的容量。

传统的单卡训练模式已无法满足前沿大模型的需求。

DeepSpeed: 为超大规模AI模型而生的分布式训练框架

- 使分布式训练和推理变得简单、高效。
- 支持具有数万亿参数的密集或稀疏模型训练。
- 实现出色的系统吞吐量，并有效扩展到数千个GPU。

吞吐量提升 (Throughput Improvement)

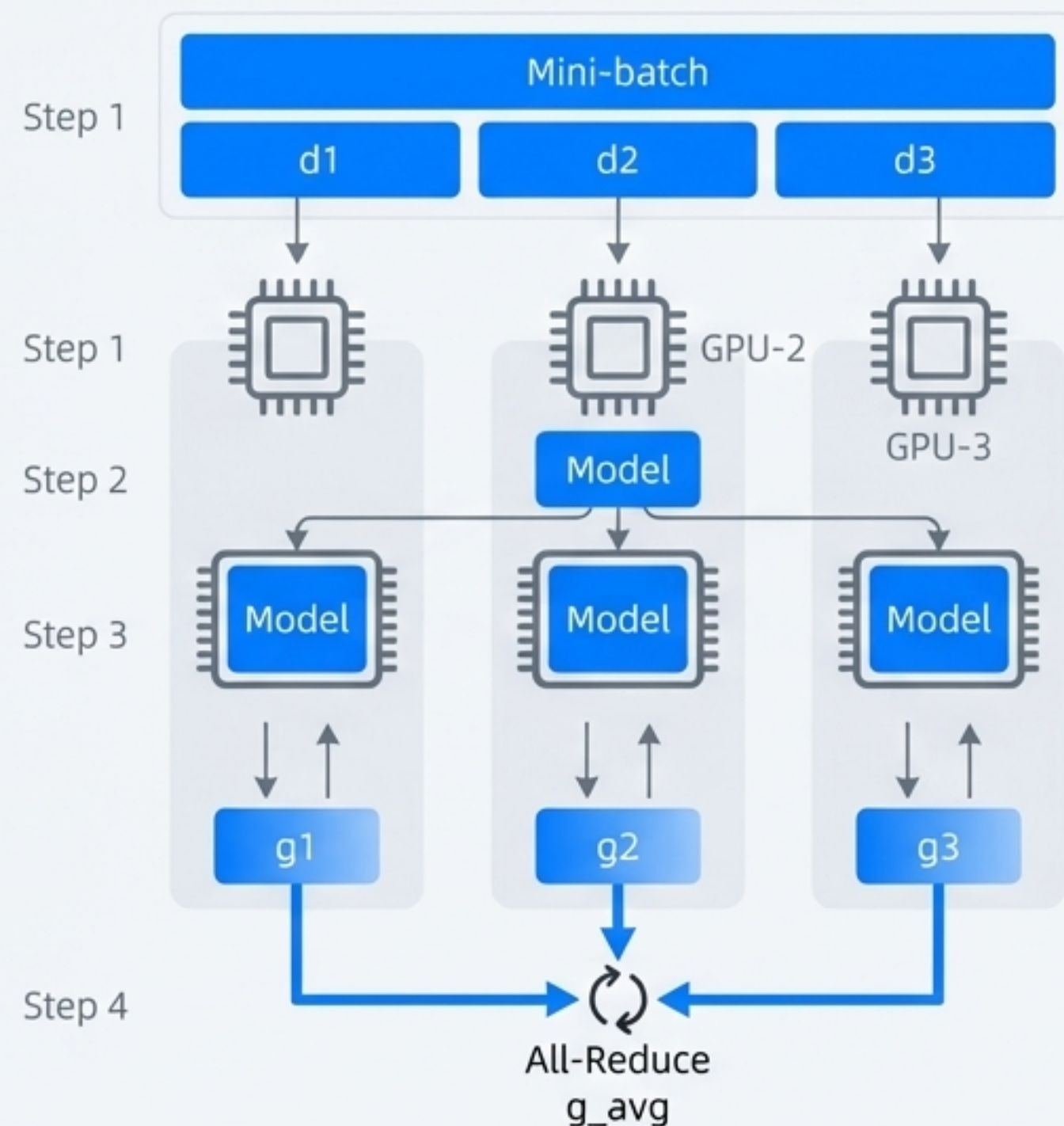


基础并行策略：数据并行（DDP）

核心思路：“模型复制，数据切分” – 每个GPU都有一份完整的模型副本，但只处理一部分数据。

How it Works

1. **分发（Scatter）**：将一个mini-batch的数据切分到多个GPU上。
2. **复制（Replicate）**：在每个GPU上复制完整的模型。
3. **前向/后向（Forward/Backward）**：每个GPU独立计算自己数据子集的梯度。
4. **聚合（Reduce）**：使用All-Reduce操作同步所有GPU的梯度，确保模型权重更新一致。



关键局限（Key Limitation）

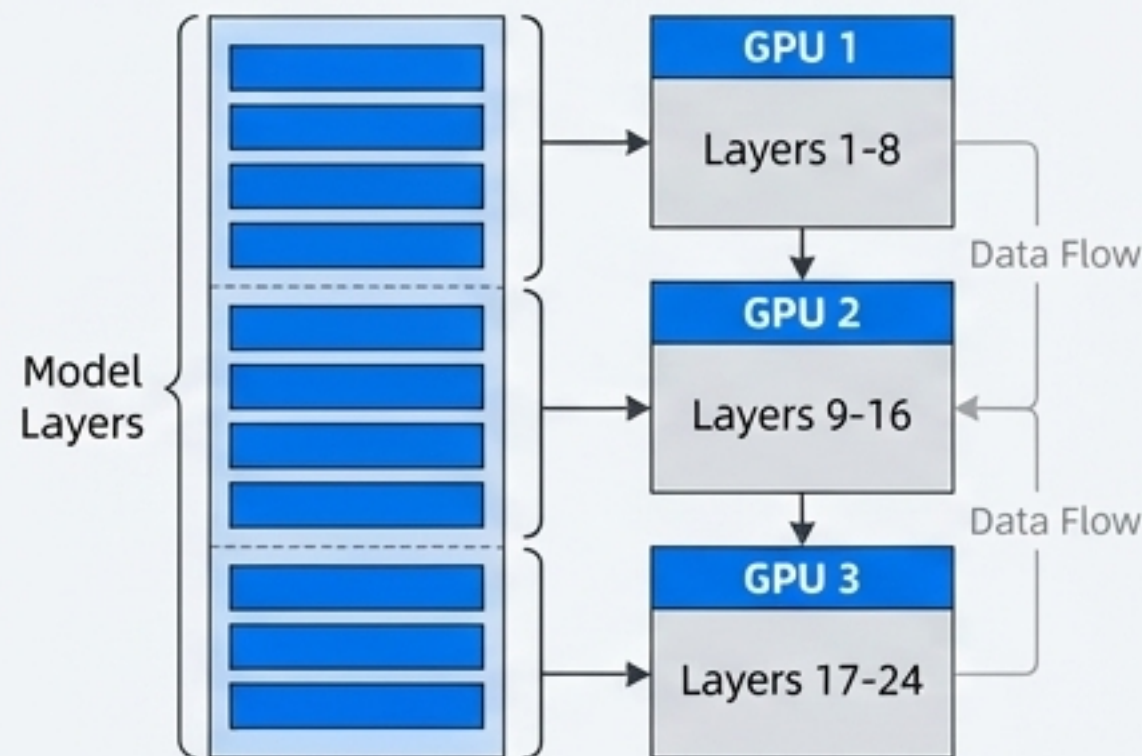
数据并行模式下，模型本身的大小必须能装入单张GPU显存。对于千亿、万亿参数的模型，此路不通。

进阶并行策略：模型并行（Model Parallelism）

核心思维：“**模型切分**” - 当模型无法放入单个GPU时，将模型本身拆分到多个GPU上。

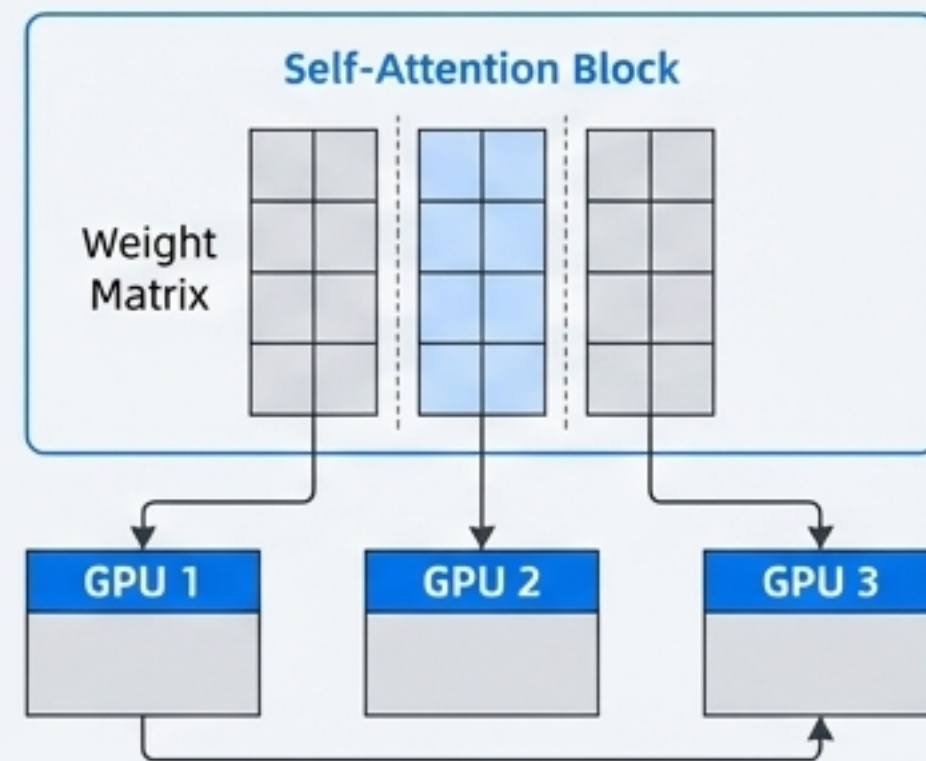
1. 流水线并行（Pipeline Parallelism）

- **类比**：工厂流水线。
- **原理**：将模型的**不同层**（Layers）放置在不同的GPU上。GPU 1处理Layer 1-8，GPU 2处理Layer 9-16，以此类推。
- **挑战**：会产生“**流水线气泡**”（Bubble），导致GPU空闲，降低效率。



2. 张量并行（Tensor Parallelism）

- **类比**：多个厨师协同处理一个巨大食材。
- **原理**：将模型**单个层**（例如一个巨大的`nn.Linear`或自注意力模块）内部的**权重矩阵**进行切分，分布到多个GPU上协同计算。



DeepSpeed核心创新：ZeRO (Zero Redundancy Optimizer)

Problem

数据并行中的隐藏问题：内存冗余

在DDP模式下，尽管数据被切分，但每个GPU仍然保留了三份完整的模型相关数据：

- 优化器状态 (Optimizer States)：如Adam优化器的动量和方差（通常是模型参数大小的2倍，FP32下是8倍）。
- 梯度 (Gradients)：与模型参数大小相同。
- 模型参数 (Model Parameters)。

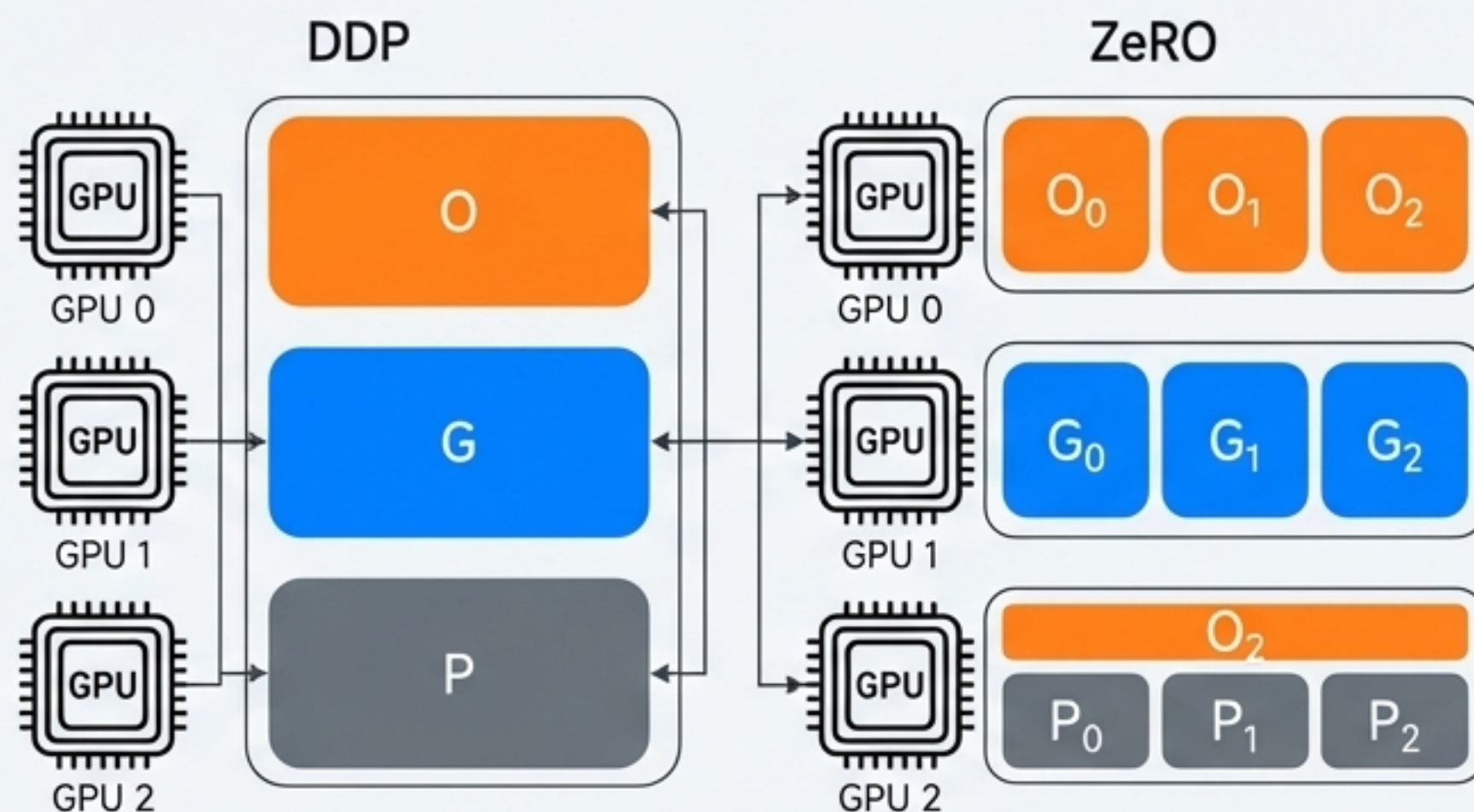
对于大模型，优化器状态是显存消耗的巨头！

Solution

ZeRO的解决方案：化整为零，分而治之

对优化器状态、梯度和参数进行分区 (Partitioning)，每个GPU只保留其中一部分，从而消除冗余，大幅节省显存。

DDP vs. ZeRO Diagram

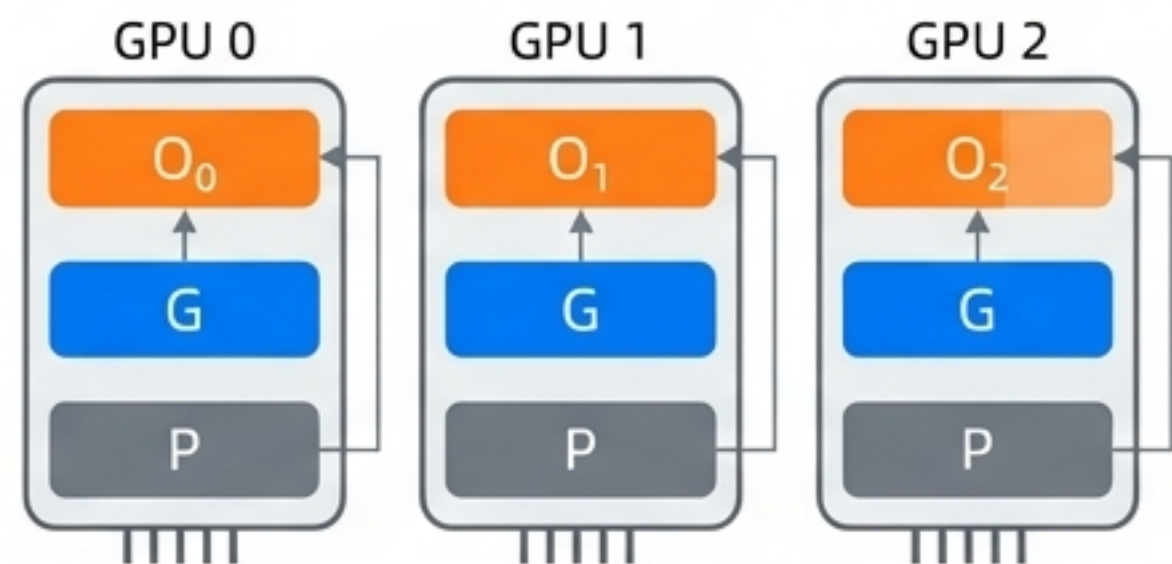


ZeRO详解：Stage 1 & Stage 2

ZeRO-1: 对优化器状态进行分区 (Partition Optimizer States)

原理：将优化器状态（如Adam的动量和方差）均匀切分到所有数据并行的GPU上。每个GPU只负责更新属于自己的那部分参数对应的优化器状态。

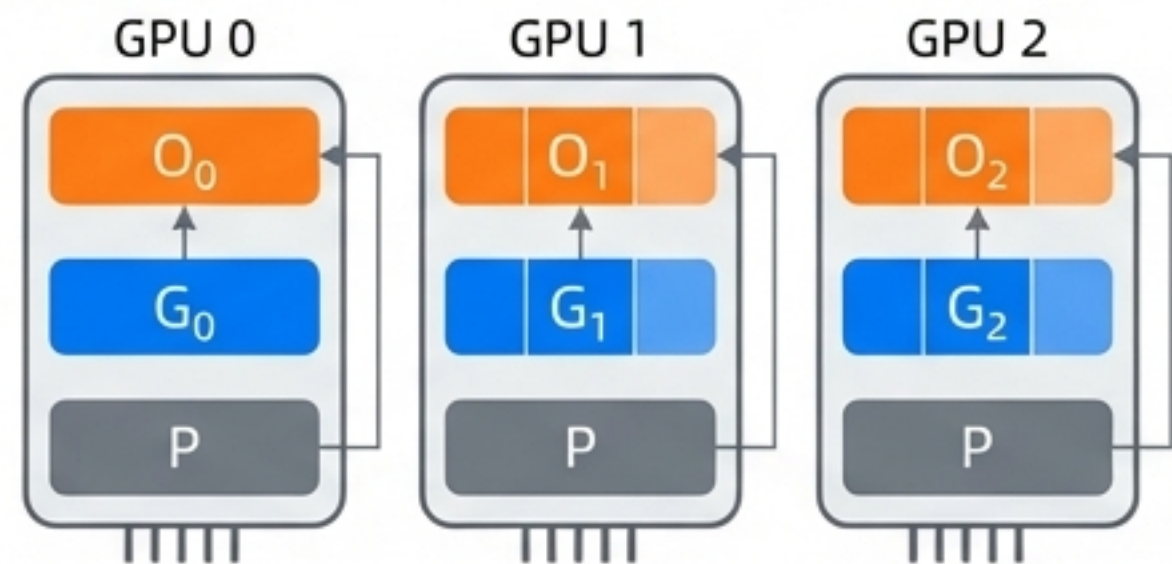
效果：消除优化器状态的冗余，大幅减少显存占用（通常可节省4倍的模型参数显存）。梯度和模型参数仍然是完整的副本。



ZeRO-2: 对优化器状态和梯度进行分区 (Partition Gradients)

原理：在Stage 1的基础上，进一步对梯度也进行分区。在反向传播过程中，每个GPU计算出的梯度在All-Reduce后，只保留自己负责更新的那一部分，其余的丢弃。

效果：进一步消除梯度冗余，显存节省量更大（通常可节省8倍的模型参数显存）。



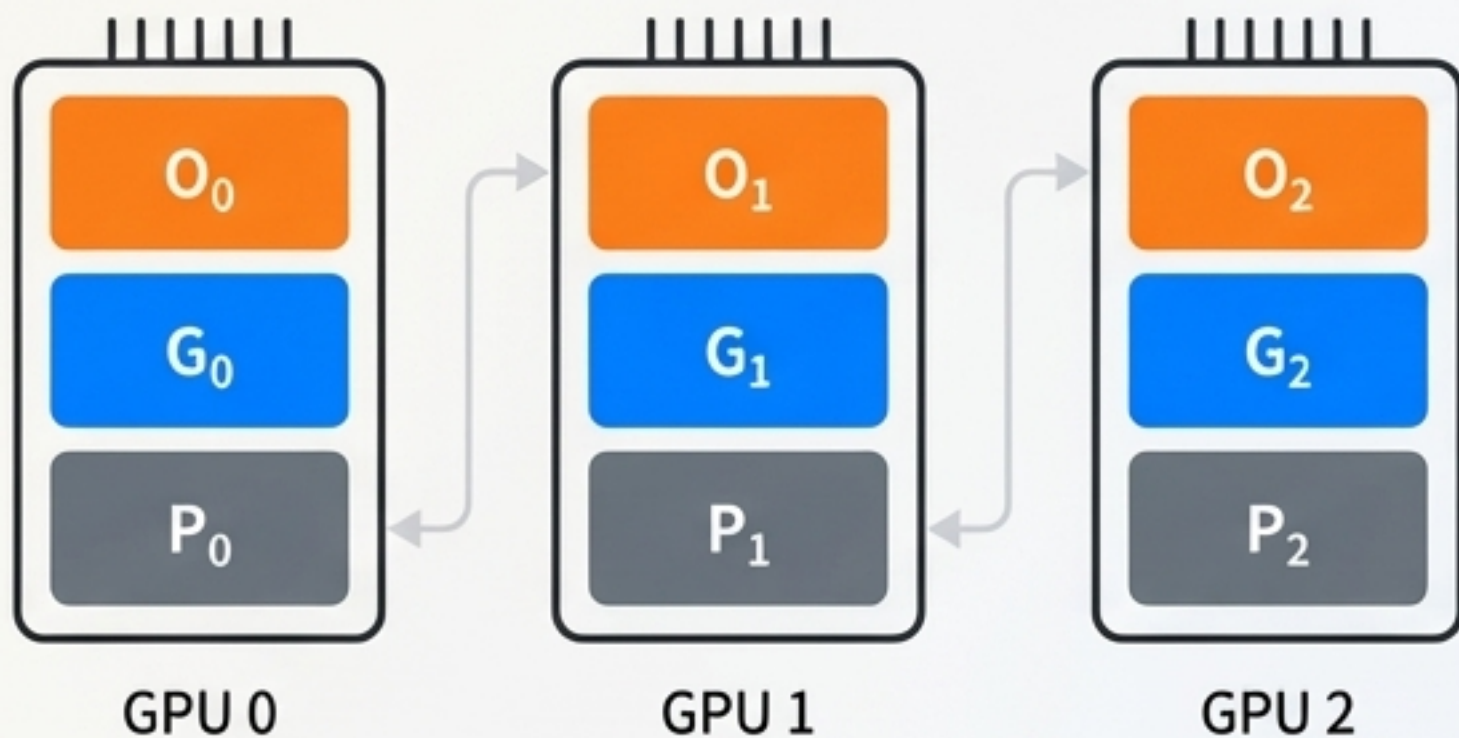
ZeRO详解：Stage 3 与 Offload 技术

ZeRO-3: 对所有模型状态进行分区 (Partition Parameters)

原理：在Stage 2的基础上，将模型参数本身也进行分区。每个GPU在任何给定时间只持有模型的一部分参数。在前向/后向传播需要时，通过通信动态地从其他GPU获取所需的参数。

效果：内存效率的极致。理论上，可以在N个GPU上训练一个比单卡显存大N倍的模型。

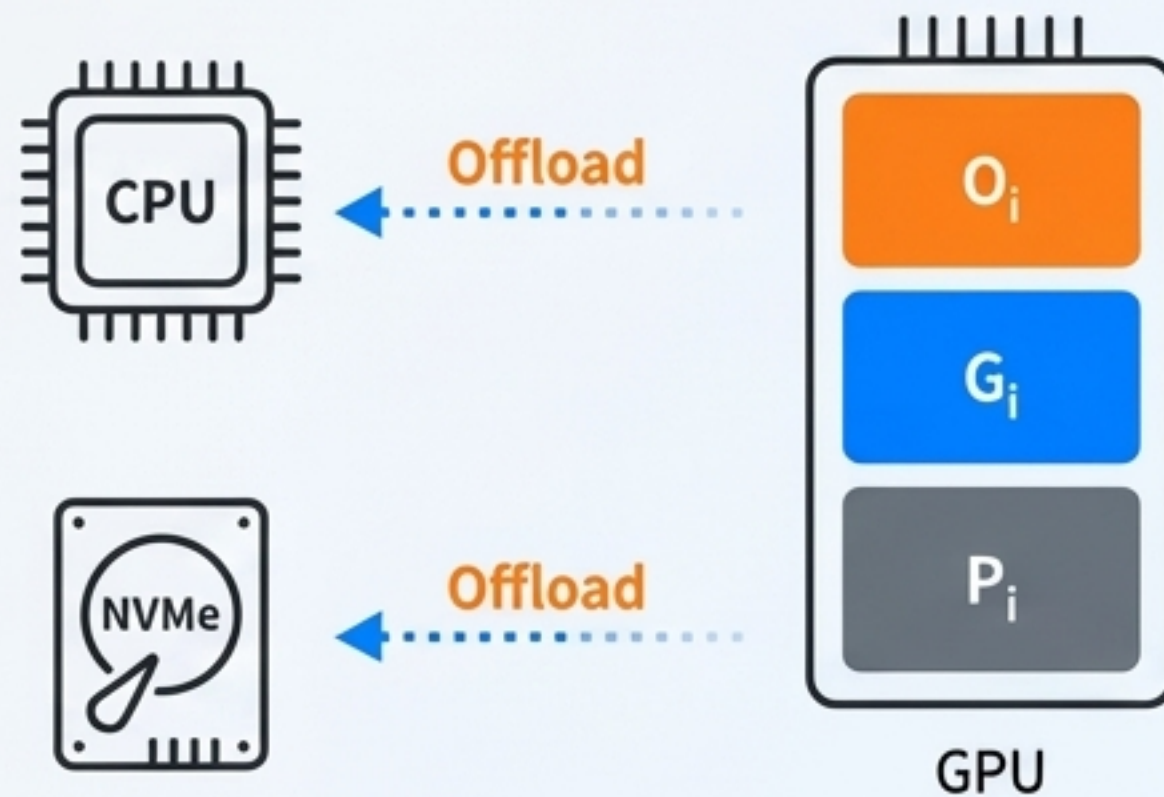
“内存减少与数据并行度和复杂度成线性关系”



突破GPU显存墙

ZeRO-Offload: 将优化器状态和梯度卸载到CPU内存。适用于GPU显存极其宝贵，但CPU内存充裕且CPU-GPU通信带宽较高的场景。

ZeRO-Infinity: 终极方案。利用NVMe固态硬盘作为内存的扩展，将不在当前计算中使用的模型参数、梯度和优化器状态卸载到CPU内存甚至NVMe硬盘，从而能够训练万亿级别的模型。



实战演练：通过`ds\_config.json`启用ZeRO

DeepSpeed将复杂的分布式策略抽象为简单的JSON配置，对用户代码的侵入性极小。

```
{
  "train_micro_batch_size_per_gpu": 1,
  "gradient_accumulation_steps": 8,
  "optimizer": {
    "type": "AdamW",
    "params": {
      "lr": 1.0e-4,
      "betas": [0.9, 0.95],
      "weight_decay": 0.1
    }
  },
  "zero_optimization": {
    "stage": 2,
    "offload_optimizer": {
      "device": "cpu",
      "pin_memory": true
    },
    "allgather_bucket_size": 5e8,
    "reduce_bucket_size": 5e8
  },
  "fp16": {
    "enabled": true
  }
}
```

只需改变这个数字即可切换ZeRO模式！(0, 1, 2, 3)

一个简单的配置块即可将优化器卸载到CPU。

轻松开启FP16混合精度，进一步加速并节省显存。

实战演练：启动DeepSpeed训练任务

```
deepspeed --num_gpus=[NUM_GPUS] [YOUR_SCRIPT.py] --deepspeed
--deepspeed_config ds_config.json
```

关键启动参数解析

Argument	Meaning	Example
master_addr	主节点的IP地址	--master_addr=10.51.97.28
master_port	主节点的端口号	--master_port=29500
nnodes	节点数（机器数量）	--nnodes=2
node_rank	节点rank，从0开始	--node_rank=0
nproc_per_node	每节点进程数（通常等于GPU卡数）	--nproc_per_node=8

注：DeepSpeed的启动器封装了复杂的分布式环境设置，让用户可以专注于模型代码本身。

总结：如何选择ZeRO策略？速度与显存的权衡

没有最优解，只有最合适解。选择取决于你的硬件限制和性能目标。



决策指南



如果你的模型和优化器可以轻松放入单卡显存：优先使用 **Stage 0 (DDP)** 或 **Stage 1** 以获得最高速度。



如果模型可以放入但优化器不行：使用 **Stage 2** 是一个很好的平衡点。



如果模型本身都无法放入单卡显存：必须使用 **Stage 3**。



如果**GPU**显存极度紧张：结合使用 **Offload** 技术，用CPU/NVMe换取运行更大模型的能力。

终局：DeepSpeed - 大模型训练的瑞士军刀

我们的旅程回顾

- 我们从大模型训练的“内存墙”挑战出发。
- 探索了基础的**数据并行**和进阶的**模型并行**策略。
- 深入剖析了DeepSpeed的核心创新 **ZeRO**，它通过消除冗余，从Stage 1到Stage 3逐步将显存优化到极致。
- 了解了如何利用 **ZeRO-Offload** 和 **Infinity** 突破硬件的物理边界。



核心启示

DeepSpeed并非单一技术，而是一个集成了数据并行、模型并行（流水线+张量）、ZeRO、Offload等多种优化技术的**综合性解决方案**。它允许工程师根据具体模型大小、硬件配置和性能需求，灵活组合这些工具，找到最高效的训练策略。